Pharmacy Inventory & Prescription Management System

Database Systems Lab (BCSE302P)

PROJECT REPORT

Fall Semester 2026-2027

Submitted by

ABHIRUP BANIK (24BCE0311)

SARVASWA BHANTI (24BCE0333)

AMOGH SHARMA (24BCE2126)

SAMI FARHAN SYED (24BCE2744)

in partial fulfillment for the award of the degree of

B. Tech

In

Computer Science and Engineering
Core



Vellore-632014, Tamil Nadu, India

School of Computer Science and Engineering

October, 2026

CONTENTS

Pharmacy Inventory & Prescription Management System

1. Short summary of the project

1.1 Overview of the Two-Database Design

1.2 Polyglot Persistence Approach

1.3 The Workflow, End to End

2. Front end and Database details

2.1 Front End Details

2.2 Database Details - Overview

2.3 Database One - PostgreSQL: The Transactional Core

2.3.1 The FEFO Query

2.3.2 The Dispensing Event / Audit Log Table

2.4 Database Two - MongoDB: The Safety Knowledge Layer

2.4.1 Sample Interaction Document

2.5 Comparison of the Two Databases

PHARMACY INVENTORY & PRESCRIPTION MANAGEMENT SYSTEM

1. SHORT SUMMARY OF THE PROJECT

The project is a pharmacy management system in which the database layer, rather than the user interface, is the centre of the design. A pharmacy has to do two things correctly at the moment a medicine is handed over: it has to deduct the right physical stock from the right batch without ever losing count, and it has to confirm that the medicine being dispensed does not conflict dangerously with the other medicines on the same prescription. These two requirements pull in different directions, and the system is built around that observation.

1.1 Overview of the Two-Database Design

When a pharmacist dispenses a medicine, PostgreSQL handles the transaction, picks the right batch and logs the event, while MongoDB checks whether the combination is safe. Each database is responsible for the part of the task the other is not well suited to.

1.2 Polyglot Persistence Approach

Polyglot persistence means picking a different database for each part of a system based on how that data actually behaves, instead of forcing everything into one engine. Here, inventory data needs strong consistency and locking because it involves real stock and real money, and it also needs a durable, ordered record of what was dispensed and when. Interaction data, by contrast, changes shape often, arrives from external medical sources and is read far more often than it is written, so a flexible document store fits it better than a rigid table definition.

The dispensing event log is deliberately kept inside PostgreSQL as an ordinary relational audit table rather than being pushed into a third engine. Writing it alongside the stock update means the audit trail is committed under the same ACID guarantees as the transaction it describes, and a single well-indexed table answers the queries the system actually needs without the added operational overhead of running and maintaining a third database.

Figure 1 shows the overall architecture, with the two engines and the responsibilities assigned to each.

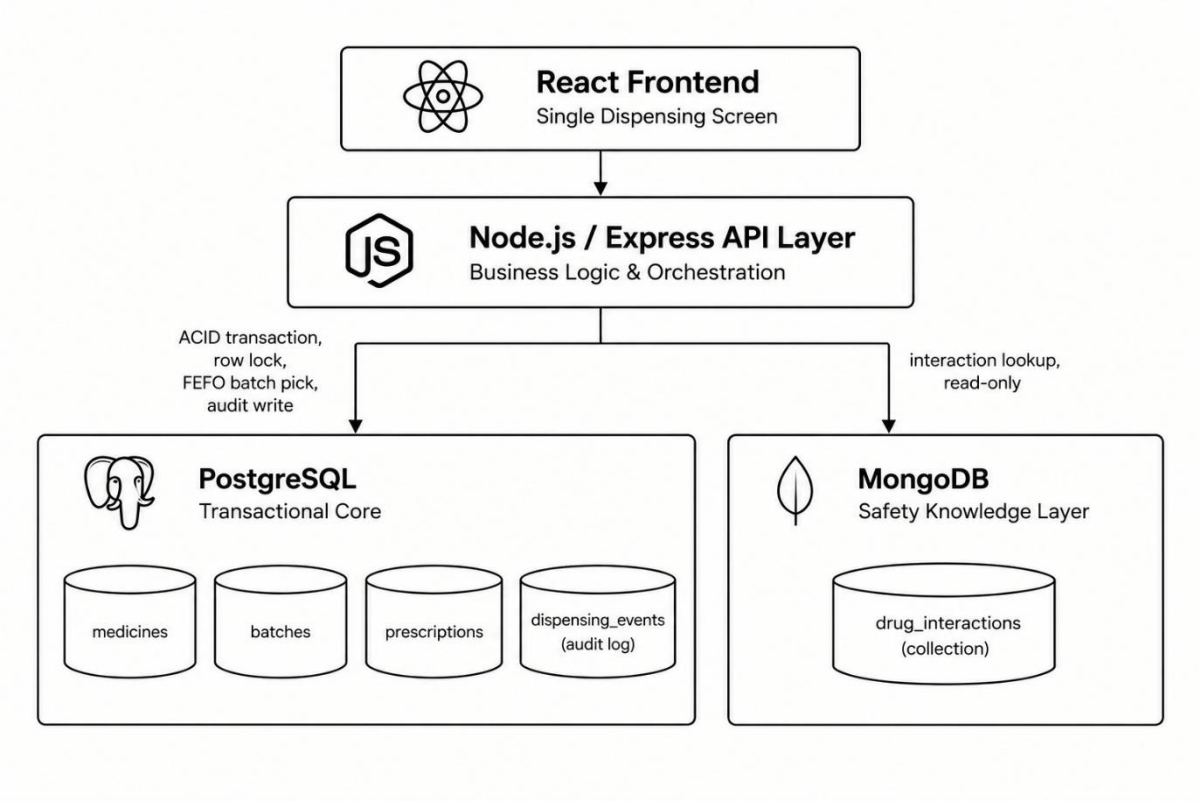


Figure 1: System architecture of the two-database polyglot persistence design

1.3 The Workflow, End to End

One dispensing action touches both databases, each for a different reason. The sequence is set out below and illustrated in Figure 2.

1. The pharmacist searches for a medicine against a prescription.
2. PostgreSQL locks the relevant batch row and selects the batch nearest to expiry, following the FEFO (first expired, first out) rule.
3. The backend queries MongoDB for interactions between all medicines on that prescription.
4. If a conflict is found, a warning is shown before the pharmacist is allowed to proceed.
5. On confirmation, PostgreSQL commits the transaction and updates the stock quantity for that batch.
6. PostgreSQL writes an entry to the dispensing_events (audit_log) table as part of the same transaction, so the event record and the stock change succeed or fail together.

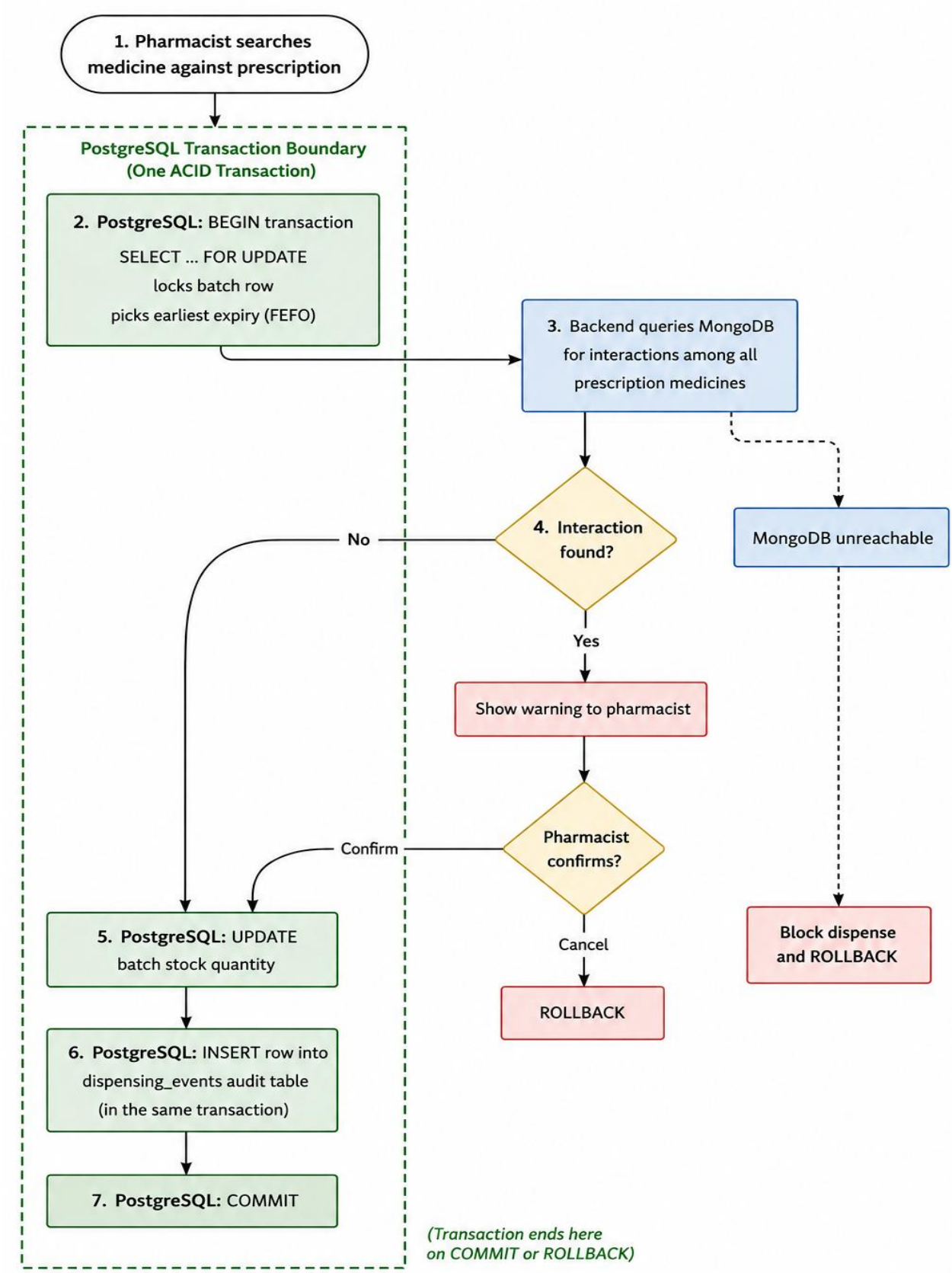


Figure 2: End-to-end dispensing workflow across PostgreSQL and MongoDB

2. FRONT END AND DATABASE DETAILS

2.1 Front End Details

The relevant Front End details are:

- A single dispensing screen: search a medicine, see batch and expiry, see any interaction warning, and confirm the dispense.
- It will be built in React once the database logic and API layer are stable, so that effort is not split across both at this stage.
- It could even begin as a command-line interface or a set of API calls tested through Postman before any user interface exists, since the logic is what is being evaluated.
- The purpose of the front end at this point is to prove the workflow end to end, not to be a polished product.

A rough layout of that single screen is shown in Figure 3.

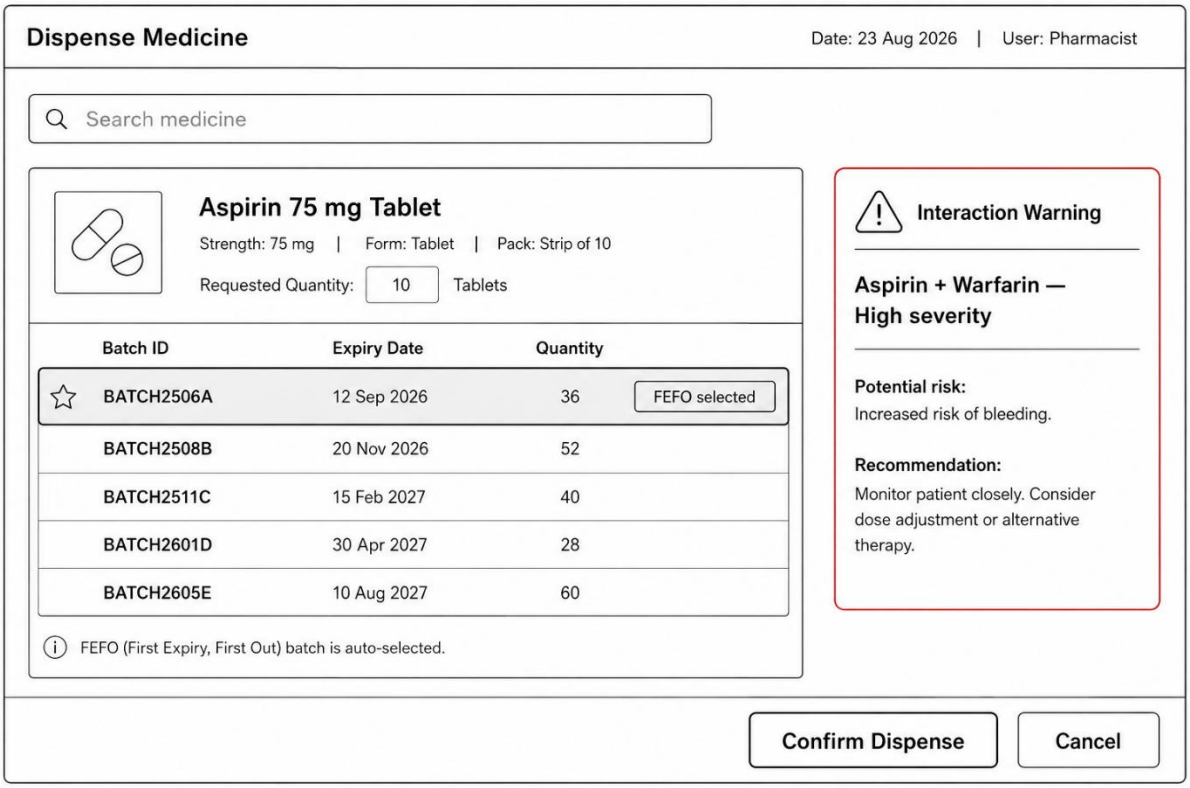


Figure 3: Wireframe of the single dispensing screen

2.2 Database Details - Overview

The system uses exactly two databases. PostgreSQL is the transactional core, holding medicines, batches, prescriptions, stock movements and the dispensing audit log. MongoDB is the safety knowledge layer, holding drug interaction records and clinical guidelines. Each is described below with the concepts it is being used to demonstrate.

2.3 Database One - PostgreSQL: The Transactional Core

Why PostgreSQL here: inventory counts and prescriptions are relational by nature and directly tied to stock and billing. If two people attempt to dispense the last unit of a batch at the same time, this concurrency issue must be prevented. PostgreSQL's ACID guarantees make this possible.

- **ACID transactions.** The whole dispense operation, from checking stock to updating quantity to writing the audit row, runs inside one transaction. If any step fails, everything rolls back and nothing is left half-updated.

- **Row-level locking.** SELECT ... FOR UPDATE locks the batch row being read. A second pharmacist trying to dispense from the same batch has to wait until the first transaction commits or rolls back.

- **Indexing for FEFO.** A B-tree index on expiry_date within each medicine's batches means the "pick the earliest expiring batch" query stays fast even as batch history grows.

- **Referential integrity.** Foreign keys tie batches to medicines and prescriptions to medicines. A batch cannot be inserted for a medicine that does not exist, and the database enforces that rather than the application code.

- **Audit logging.** A dispensing_events table records every dispense, stock adjustment and batch receipt as an append-only row, written as part of the same transaction that changes the stock.

The relational schema, including the foreign-key relationships and the audit table, is shown in Figure 4.

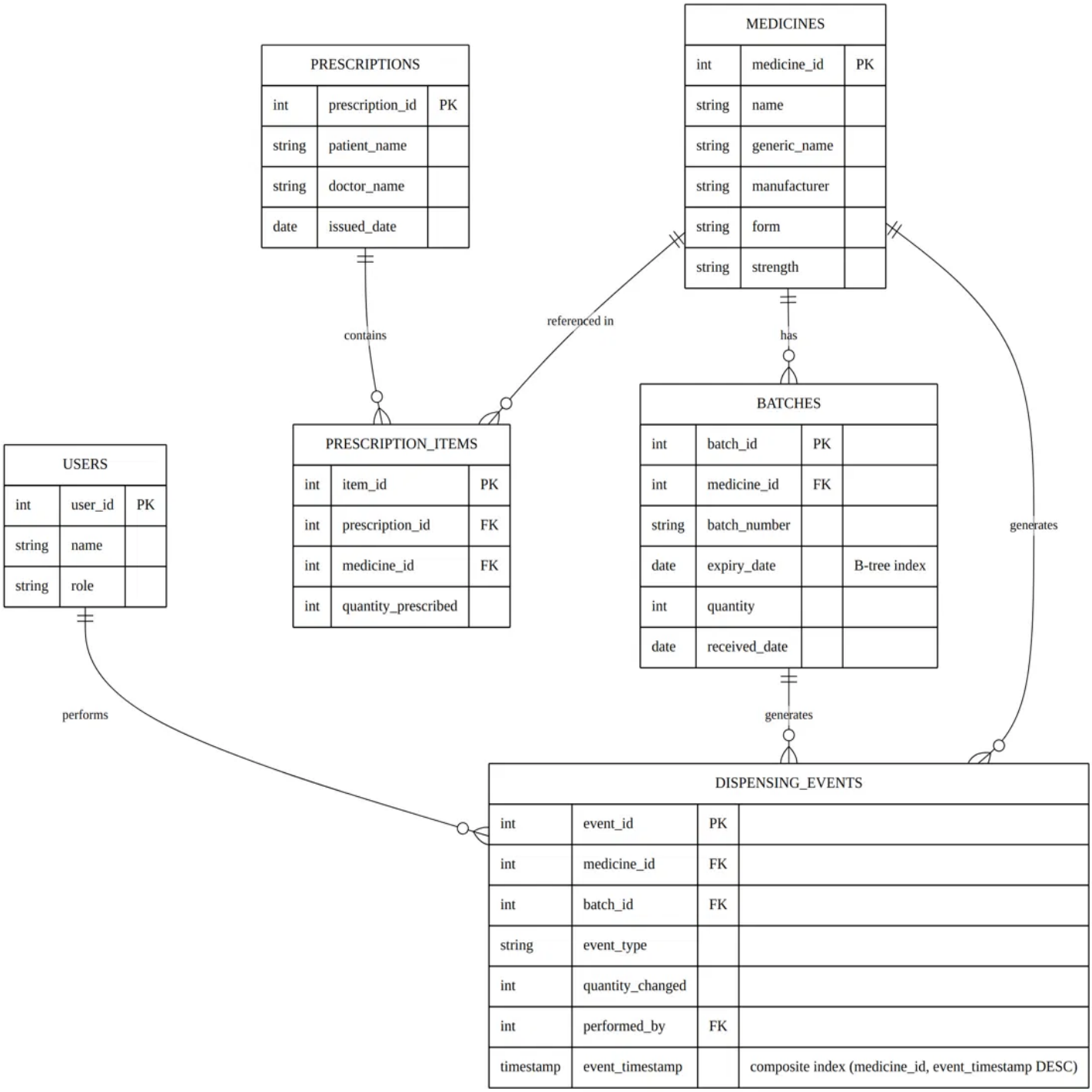


Figure 4: ER diagram of the PostgreSQL schema

2.3.1 The FEFO Query

This is the query at the centre of the transactional design. It is also the query that needs the lock, because two transactions could otherwise both read the same row before either commits.

```
SELECT * FROM batches
WHERE medicine_id = $1 AND quantity > 0
ORDER BY expiry_date ASC
LIMIT 1
FOR UPDATE;
```

Figure 5 traces what happens when two pharmacists attempt to dispense from the same batch at the same instant, and shows how the row lock serialises them.

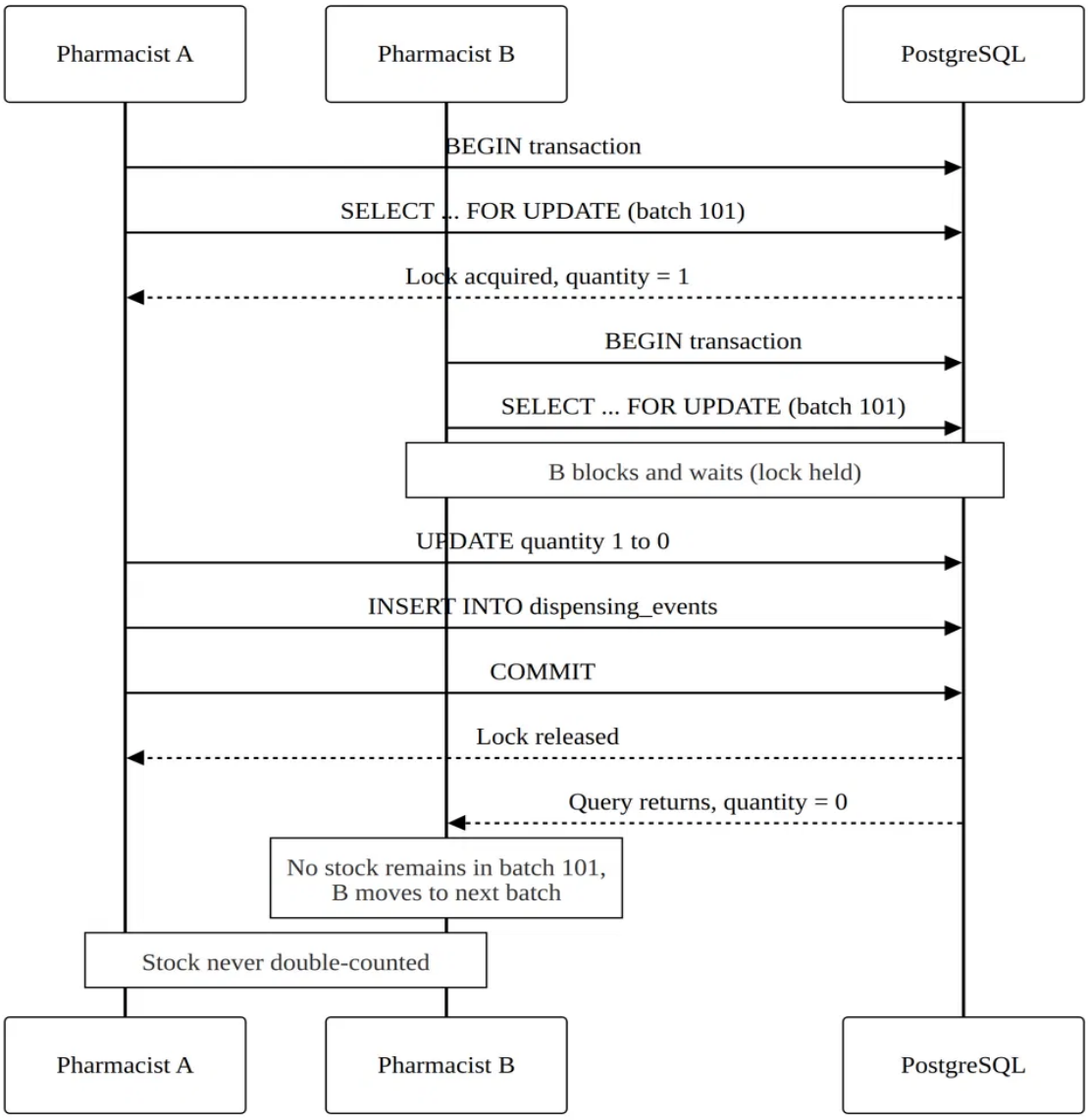


Figure 5: Concurrent dispensing handled by row-level locking (SELECT ... FOR UPDATE)

2.3.2 The Dispensing Event / Audit Log Table

Every dispense, stock adjustment and batch receipt generates an event that has to be recorded. Rather than introducing a separate database engine for this, the log is implemented as an ordinary PostgreSQL table written to as part of, or immediately after, the dispensing transaction commits. The table is deliberately narrow and append-only:

```
dispensing_events (
    event_id          BIGSERIAL PRIMARY KEY,
    medicine_id       INT       REFERENCES
medicines(medicine_id),
    batch_id          INT       REFERENCES batches(batch_id),
    event_type        VARCHAR(30) NOT NULL,
    quantity_changed  INT       NOT NULL,
    performed_by      INT       REFERENCES users(user_id),
    event_timestamp   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

An index on (medicine_id, event_timestamp DESC) supports the query the system actually asks of this table: “show me the recent history for this medicine, most recent first”, without a full table scan. A secondary index on event_timestamp alone supports time-window reporting such as “most dispensed medicines this week”, supporting reporting queries such as identifying the most dispensed medicines in a given week.

This table is not the source of truth for current stock; the batches table is. The audit log is a record of what happened over time, and because it is written inside the same transaction, it can never disagree with the stock change it describes.

2.4 Database Two - MongoDB: The Safety Knowledge Layer

Why MongoDB here: drug interaction data comes from external medical sources and does not have a fixed shape. Some interactions have dosage thresholds, some do not. Some guidelines have five fields, some have twelve. Forcing this into rigid tables means constant schema migrations. A document model accommodates this variability without requiring schema changes.

- **Schema flexibility.** Each document in the drug_interactions collection can carry different fields depending on what is known about that pair of drugs, without altering a table definition.

- **Document model versus relational.** One interaction record maps to one real-world fact. There is no need to join across multiple tables just to answer "do these two drugs conflict?"

- **Indexing on drug pairs.** A compound index on drug_a and drug_b keeps the interaction check fast, since this query runs on every single dispense.

- **Read-heavy workload.** The collection is updated only occasionally, as new interactions are published, but read on nearly every transaction. MongoDB handles that read pattern well.

2.4.1 Sample Interaction Document

```
{
    drug_a:         "Aspirin",
    drug_b:         "Warfarin",
    severity:       "high",
    effect:         "Increased bleeding risk",
    recommendation: "Avoid combination"
}
```

Figure 6 contrasts this document structure with the equivalent normalised relational representation, illustrating why the document model was preferred for this data.

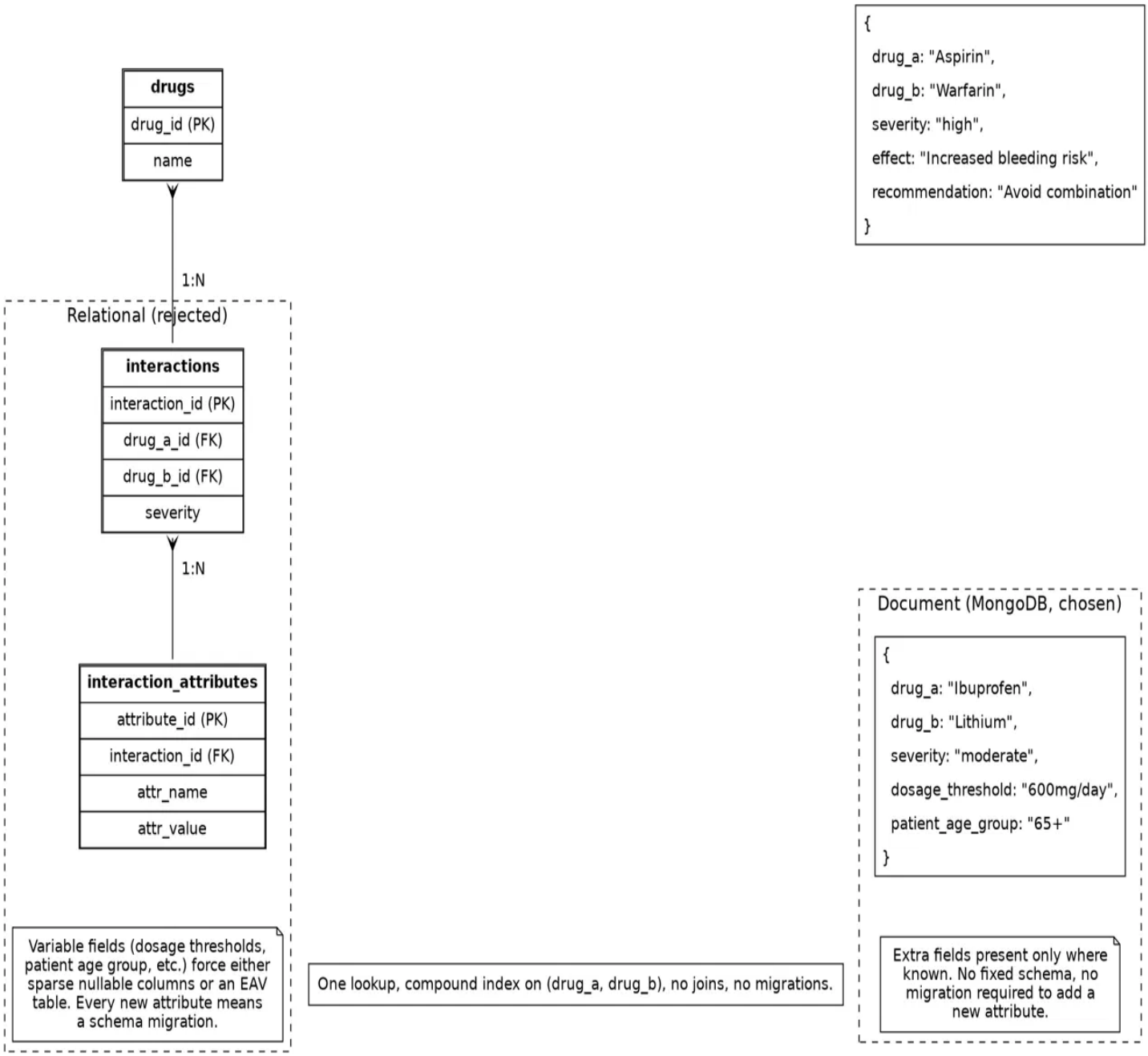


Figure 6: Document model versus relational model for drug interaction data

2.5 Comparison of the Two Databases

The two engines are placed side by side below, on the dimensions that motivated the choice.

Aspect	PostgreSQL	MongoDB
Data shape	Rigid, relational, strongly typed	Flexible, document-oriented (BSON)
Optimised for	Consistency, transactions, locking, referential integrity	Read-heavy access, schema-variable data, fast lookups
CAP leaning	Consistency first	Consistency with tunable read preference; availability-friendly
Used for	Medicines, batches, prescriptions, stock updates and the dispensing audit log	Drug interactions and clinical guidelines
Write pattern	Transactional writes inside ACID boundaries, plus append-only audit rows	Occasional writes when new interaction data is published
Read pattern	Point lookups and indexed range scans (FEFO, recent history per medicine)	Compound-index lookups on drug pairs, run on nearly every dispense